From the example above we can see that extending `SuperClass` is fine, but overloading the `final` method `doThing()` emits an error. Now we'll attempt to extend a `final` class…

```
final class SuperClass
{
}

class SubClass extends SuperClass
{
}

Output:
Fatal error: Class SubClass cannot extend final class SuperClass
```

… with predictable (fatal) results.

Declaring classes or methods `final` has its place, which is obviously when extension or overriding must be prevented. Doing so, however, might conflict with a fundamental principle of OOP, something called the "Open/Closed Principle," which mandates a module (or class, in this case) must be open for extension but closed for modification. We cover these principles in detail in Chapter 10, "Programming with Objects." If you're wondering, it is common to see `final` used for classes designed to test other classes.

Traits

Some object-oriented languages allow inheritance from multiple parent classes but PHP is not one of them. The object model of PHP is one of *single inheritance*. To mitigate this limitation (although it can be argued that single inheritance is a limitation that exists for good reasons), *traits* were added to the language.

A trait declaration looks a lot like a class declaration:

```
trait Engine
{
    protected int $fuelLevel = 0;

    public function printFuelLevel(): void
    {
        print "There are {$this->fuelLevel} litres remaining!" . PHP_EOL;
    }
```